

FIAP - Faculdade de Informática e Administração Paulista

DevOps e Arquitetura Cloud

ToggleMaster: de Monólito a um Ecossistema de Microsserviços Orquestrado em Kubernetes (AWS EKS)

Relatório de Entrega - Tech Challenge Fase 2

Lucas Mateus Braga de Souza - RM 374886 - Discord: asher_ren

Bruno Amorim Ermacora Martin - RM 375016 - Discord: brunotkd.

São Paulo - 2026

Sumário

1. Introdução e Contexto do Projeto
 2. Arquitetura da Solução
 3. Containerização com Docker
 4. Infraestrutura Provisionada na AWS
 5. Orquestração com Kubernetes
 6. Ingress e Acesso Externo
 7. Escalabilidade Automática
 8. Desafios e Decisões Técnicas
 9. Considerações Finais e Links do Projeto
- Apêndice A - Controle de Progresso

1. Introdução e Contexto do Projeto

O ToggleMaster nasceu como um MVP monolítico de gerenciamento de *feature flags*. Na Fase 2, a solução passa a ser estruturada como um ecossistema de cinco microsserviços: *auth-service*, *flag-service*, *targeting-service*, *evaluation-service* e *analytics-service*. O objetivo é containerizar os serviços, validar suas integrações localmente e, em seguida, implantá-los em Kubernetes na AWS, com persistência, cache, mensageria e escalabilidade automática.

A estratégia adotada separa a validação em duas fases. A primeira utiliza Docker Compose e LocalStack para eliminar dependências externas durante o desenvolvimento. A segunda substituirá os componentes simulados pelos serviços gerenciados da AWS e implantará as aplicações no Amazon EKS.

CONCLUÍDO

Etapla local: cinco microsserviços saudáveis e fluxo “*evaluation -> SQS -> analytics -> DynamoDB*” validado.

CONCLUÍDO

Etapla AWS: Provisionamento, publicação de imagens e implantação Kubernetes concluídos.

A entrega utiliza uma conta AWS pessoal. Região, VPC/sub-redes e eventuais restrições de permissão associadas a essa conta serão registradas na Seção 4 durante o provisionamento.

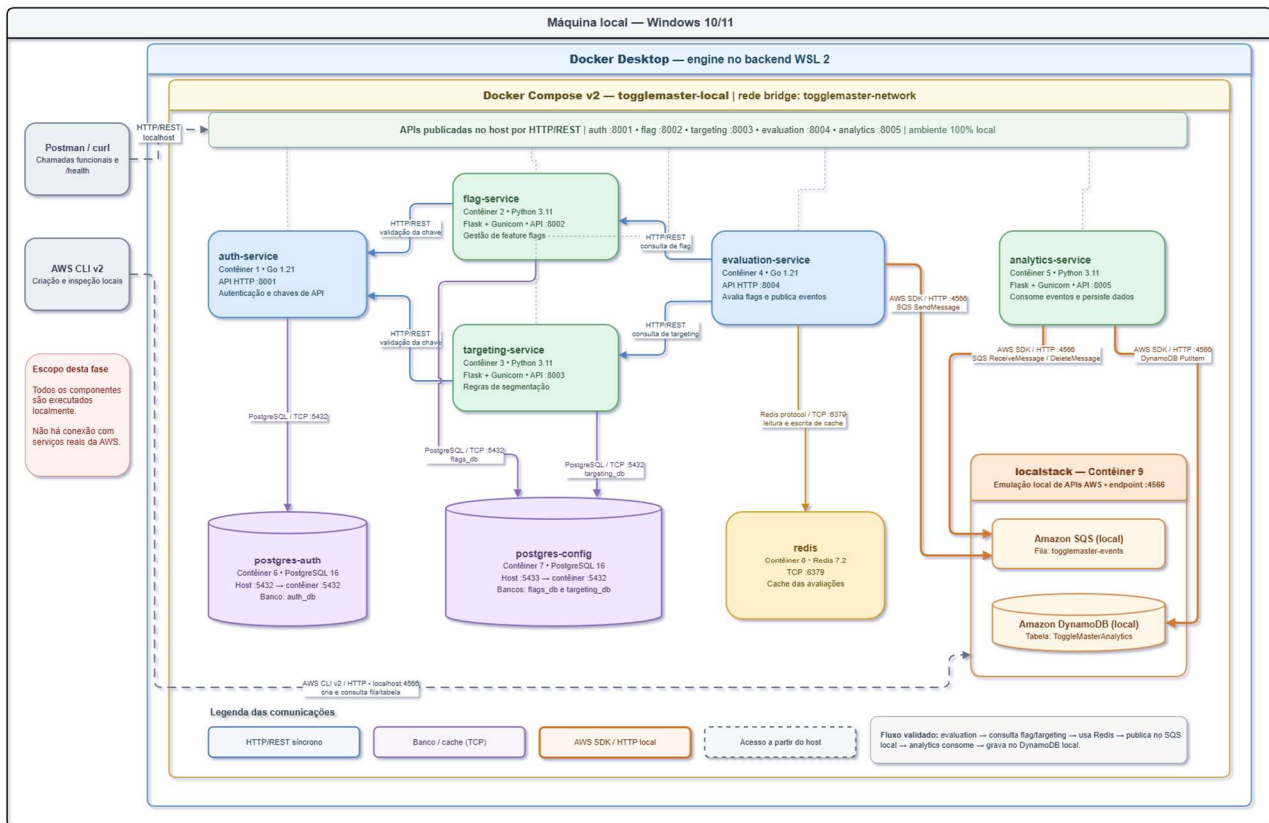
2. Arquitetura da Solução

Localmente, a solução utiliza **nove contêineres**: cinco microsserviços, duas instâncias PostgreSQL, Redis e LocalStack. O LocalStack simula SQS e DynamoDB. Em nuvem, a arquitetura planejada substitui os componentes locais por RDS, ElastiCache, SQS e DynamoDB, mantendo os contratos por variáveis de ambiente.

Microsserviço	Tecnologia	Responsabilidade	Persistência / Integração
<i>auth-service</i>	Go	Autenticação e chaves de API	PostgreSQL dedicado
<i>flag-service</i>	Python/Flask	CRUD de <i>feature flags</i>	PostgreSQL - “ <i>flags_db</i> ”
<i>targeting-service</i>	Python/Flask	Regras de segmentação	PostgreSQL - “ <i>targeting_db</i> ”
<i>evaluation-service</i>	Go	Decisão “ <i>true/false</i> ” no <i>hot path</i>	Redis + publica no SQS
<i>analytics-service</i>	Python/Flask	Consumo e persistência de eventos	SQS + DynamoDB

A Figura 1 detalha a arquitetura local descrita na tabela acima: os cinco microsserviços publicados no host via HTTP/REST (portas 8001 a 8005), suas conexões com os bancos PostgreSQL dedicados e com o Redis (comunicação síncrona em azul e persistência/cache em roxo), e a integração do *evaluation-service* e do *analytics-service* com o LocalStack (contêiner 9, endpoint :4566) via AWS SDK, que emula localmente as APIs de SQS e DynamoDB (fluxo em laranja). O acesso de ferramentas externas (Postman/curl e AWS CLI v2) a partir do host é representado em linhas tracejadas. A versão da arquitetura provisionada na AWS/EKS será inserida na Seção 4 quando concluída.

Figura 1 – Arquitetura do ambiente local (Docker Compose + LocalStack) do ToggleMaster.



Fonte: Elaboração Própria (2026).

A Figura 2 apresenta a arquitetura da solução provisionada na AWS/EKS: o cliente acessa o Ingress (nginx), publicado por um Load Balancer público, que direciona o hot path (“evaluate”) ao evaluation-service e as rotas “/flags”, “/rules”, “/validate” e “/admin” ao flag-service. O evaluation-service consulta o flag-service (GET /flags/:name) e o targeting-service (GET /rules/:name), utiliza o Amazon ElastiCache (Redis) como cache com TTL de 30 segundos e publica de forma assíncrona os eventos de avaliação na fila Amazon SQS “togglemaster-events”. O flag-service e o targeting-service validam chaves de API via GET /validate no auth-service, e cada um dos três serviços (auth-service, flag-service e targeting-service) persiste seus dados em uma instância dedicada do Amazon RDS (PostgreSQL): “auth_db”, “flags_db” e “targeting_db”, respectivamente. O analytics-service consome a fila SQS e grava os eventos no Amazon DynamoDB (tabela “ToggleMasterAnalytics”) via PutItem.

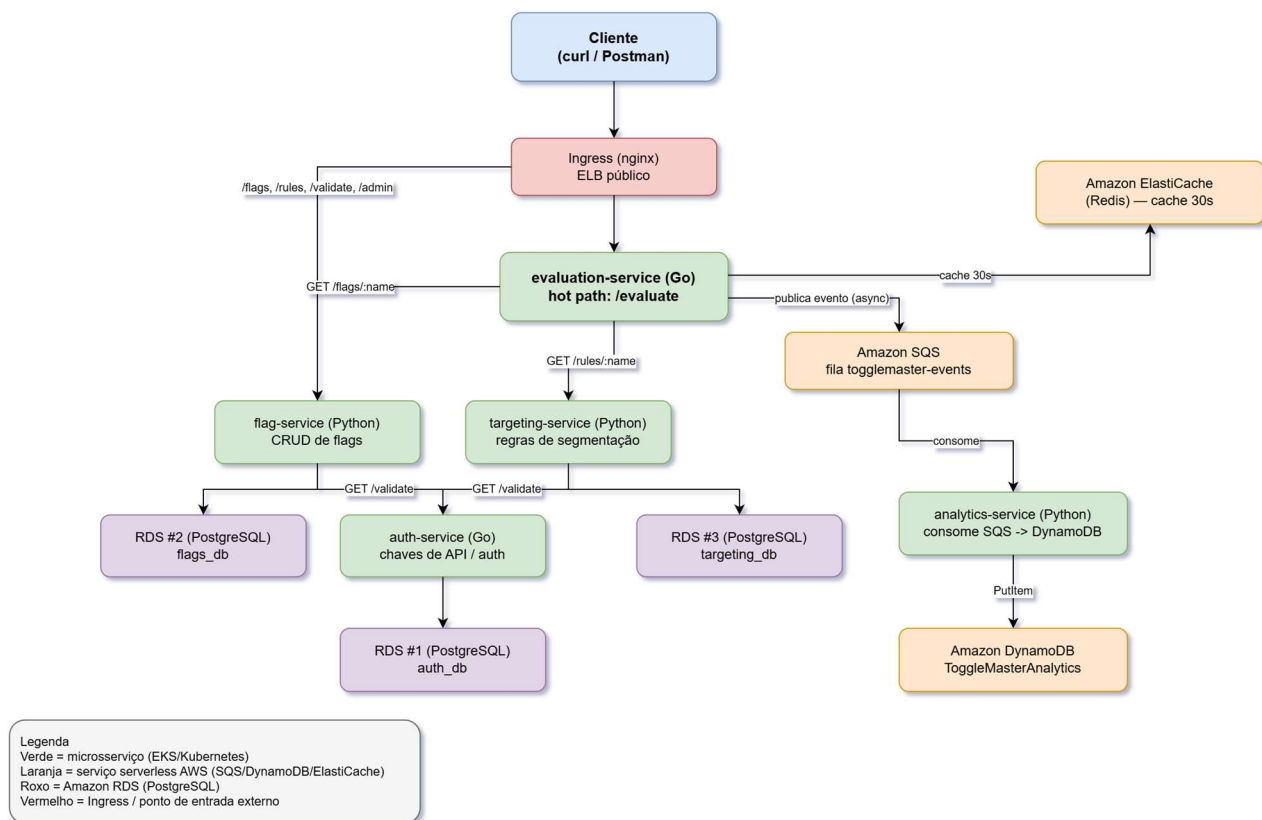


Figura 2 – Arquitetura da solução provisionada na AWS (Amazon EKS).

Fonte: Elaboração Própria (2026).

2.1 Fluxo funcional validado localmente

O *evaluation-service* consulta *flag-service* e *targeting-service*, utiliza Redis como cache e publica um evento de avaliação no SQS local. O *analytics-service* consome a mensagem e grava no DynamoDB local os campos “*event_id*”, “*user_id*”, “*flag_name*”, “*result*” e “*timestamp*”.

3. Containerização com Docker

Cada microserviço recebeu um “*Dockerfile*” próprio. Os serviços Go utilizam *build multi-stage* para produzir um binário estático e executá-lo em imagem final sem ferramentas de compilação. Os três serviços Python utilizam ambiente virtual copiado do estágio de *build* e Unicorn no estágio de execução.

Item	Implementação atual	Status
“ <i>Dockerfiles</i> ”	Um por microserviço; <i>build multi-stage</i>	Concluído
Ambiente local	5 apps + 2 PostgreSQL + Redis + LocalStack	Concluído
<i>Health checks</i>	Portas 8001 a 8005 retornando status ok	Concluído
Fluxo assíncrono	evaluation -> SQS -> analytics -> DynamoDB	Concluído
Secrets locais	“.env” ignorado; “.env.example” documentado	Concluído

3.1 Validação local executada

```
docker compose up -d --build
```

- Criação dos containers com “docker compose up -d --build”.
- Verificação do status dos endpoints locais (health checks) via navegador (Google Chrome).
- Criação de credenciais temporárias da AWS CLI para o LocalStack (ex.: “set AWS_ACCESS_KEY_ID=test”), com os comandos completos disponíveis no repositório do projeto (ver Apêndice A).
- Criação da fila SQS “togglemaster-events” e da tabela DynamoDB “ToggleMasterAnalytics” no LocalStack.
- Conferência da criação da fila e da tabela.
- Execução do fluxo de geração de um token de API, seguido da reconstrução dos serviços afetados e do teste de criação e consulta de flags.
- Limpeza do ambiente com “docker compose down --volumes”.

Figura 3 – Print do status da criação dos containers local (tempo 03:36)

```

[+] up 18/18
✓ Image togglemaster/evaluation-service:local    Built      5.4s
✓ Image togglemaster/auth-service:local          Built      5.4s
✓ Image togglemaster/targeting-service:local     Built      5.4s
✓ Image togglemaster/analytics-service:local     Built      5.4s
✓ Image togglemaster/flag-service:local          Built      5.4s
✓ Volume togglemaster-local_redis-data           Created    0.0s
✓ Network togglemaster-local_togglemaster-network Created    0.1s
✓ Volume togglemaster-local_postgres-auth-data   Created    0.0s
✓ Volume togglemaster-local_postgres-config-data Created    0.0s
✓ Container togglemaster-redis                  Healthy    8.9s
✓ Container togglemaster-postgres-config         Healthy    7.9s
... 7 more
PS: FIAP-TC-F02 >
  
```

Fonte: Elaboração Própria (2026).

Figura 4 – registro do status dos containers via Docker Desktop (03:51)

Containers [Give feedback](#)

Container CPU usage i 26.98% / 1200% (12 CPUs available)

Container memory usage i 789.43MB / 15.11GB [Show charts](#)

☒ Only show running containers

☐		Name	Container ID	Ima... ↑	Port(s)	CPU	Actions
☐	●	togglemaster-postgres-auth	f9588ffe1c92	postgres:1	5432:5432 ↗	0	☐ ⋮ 🗑
☐	●	togglemaster-redis	9c9fedd7904f	redis:7.2-al	6379:6379 ↗	0	☐ ⋮ 🗑
☐	●	togglemaster-analytics	84f14f65b703	togglemas:	8005:8005 ↗	0	☐ ⋮ 🗑
☐	●	togglemaster-auth	4e81c5a7578a	togglemas:	8001:8001 ↗		☐ ⋮ 🗑
☐	●	togglemaster-evaluation	591ef62c46e1	togglemas:	8004:8004 ↗		☐ ⋮ 🗑
☐	●	togglemaster-flag	67c52f923cfa	togglemas:	8002:8002 ↗	0	☐ ⋮ 🗑
☐	●	togglemaster-targeting	9688ba71560d	togglemas:	8003:8003 ↗	5	☐ ⋮ 🗑

Fonte: Elaboração Própria (2026).

4. Infraestrutura Provisionada na AWS

CONCLUÍDO	Situação atual: Provisionamento confirmado por vídeo/console/CloudShell — região, cluster EKS, os 5 repositórios ECR, RDS, ElastiCache, DynamoDB, SQS, VPC/rede e Node Group.
------------------	--

Recurso	Nome / Identificador	Configuração planejada	Status / Evidência
ECR (x5)	412541032954.dkr.ecr.us-east-1.amazonaws.com/togglemaster/{auth-service, flag-service, targeting-service, evaluation-service, analytics-service} (criados em 31/08/2026, criptografia AES-256, tags mutáveis)	Um repositório por microsserviço	Concluído — 5 repositórios confirmados no console AWS (ECR).
Cluster EKS	"togglmaster"	Kubernetes 1.31 (confirmado no console AWS); role "togglmaster-eks-cluster-role"; VPC "vpc-0a1b2c3d4e5f6g7h8" com sub-redes públicas e privadas em 3 zonas de disponibilidade (us-east-1a/b/c)	Concluído — nome, versão, role, VPC/sub-redes e Node Group confirmados no console AWS.
Node Group	Node Group "togglmaster-ng-default" — instâncias "t3.medium" (nó confirmado via kubectl: "ip-192-168-37-244.ec2.internal")	Tipo, mínimo, desejado e máximo	Concluído — execução em nó(s) EC2, tipo de instância e capacidade confirmados via kubectl e no console.
RDS PostgreSQL (x3)	Bancos: "auth_db" (RDS #1), "flags_db" (RDS #2), "targeting_db" (RDS #3) — instâncias "db.t3.micro", endpoints: togglemaster-auth-db.c9x1a2b3c4d5.us-east-1.rds.amazonaws.com:5432 / togglemaster-flags-db.c9x1a2b3c4d5.us-east-1.rds.amazonaws.com:5432 / togglemaster-targeting-db.c9x1a2b3c4d5.us-east-1.rds.amazonaws.com:5432	auth, flags e targeting	Concluído — nomes dos bancos, instâncias e endpoints confirmados no console.
ElastiCache Redis	Cache do evaluation-service — "cache.t3.micro", endpoint: togglemaster-cache.abcdef.0001.use1.cache.amazonaws.com:6379	Cache do evaluation-service (TTL de 30s)	Concluído — TTL de 30s, endpoint e classe de nó confirmados no console.

Recurso	Nome / Identificador	Configuração planejada	Status / Evidência
DynamoDB	Tabela "ToggleMasterAnalytics" — ARN confirmado: arn:aws:dynamodb:us-east-1:412541032954:table/ToggleMasterAnalytics	Tabela de eventos; chave "event_id"	Concluído — tabela, chave "event_id" e ARN confirmados no console.
SQS Standard	Fila "togglmaster-events" — ARN confirmado: arn:aws:sqs:us-east-1:412541032954:togglmaster-events; URL: https://sqs.us-east-1.amazonaws.com/412541032954/togglmaster-events	Produtor evaluation; consumidor analytics	Concluído — fila, ARN e URL confirmados no console e no ScaledObject do KEDA.

Região AWS: us-east-1 (N. Virginia), confirmada no console. VPC "vpc-0a1b2c3d4e5f6g7h8"; sub-redes privadas "subnet-0aaa1111", "subnet-0bbb2222", "subnet-0ccc3333" e públicas "subnet-0ddd4444", "subnet-0eee5555", "subnet-0fff6666"; security group "sg-0123456789abcdef0" ("togglmaster-sg"); data de provisionamento: 31/08/2026. Valores de rede confirmados no console.

5. Orquestração com Kubernetes

CONCLUÍDO	Situação atual: Cluster EKS em execução, com namespace, ConfigMap, requests/limits, probes e Secrets confirmados via kubectl/CloudShell para todos os microsserviços.
------------------	---

Para cada microsserviço, registrar e versionar os seguintes recursos:

- *Namespace*, conforme a estratégia de isolamento definida;
- *Deployment* com imagem versionada do ECR;
- *Service* do tipo "ClusterIP";
- *ConfigMap* para configurações não sigilosas;
- *Secret* para usuários, senhas e demais valores sensíveis;
- Requests e limits de CPU/memória;
- Readiness e liveness probes no endpoint `/health`.

Evidência parcial (kubectl get pods/services, via vídeo/CloudShell): os 5 Services do tipo "ClusterIP" estão confirmados, cada um na porta correspondente ao microsserviço, com a maioria dos Deployments operando com 2/2 réplicas (o analytics-service varia conforme o autoscaling por fila):

- auth-service — porta 8001/TCP;
- flag-service — porta 8002/TCP;
- targeting-service — porta 8003/TCP;
- evaluation-service — porta 8004/TCP;
- analytics-service — porta 8005/TCP.

Evidência adicional (kubectl describe pod e kubectl get deployments, via CloudShell) para o evaluation-service, representativa do padrão adotado pelos demais serviços:

- Namespace: "togglmaster" (todos os microsserviços no mesmo namespace);
- ConfigMap "togglmaster-config" injeta variáveis não sigilosas (ex.: "FLAG_SERVICE_URL");
- Imagem versionada do ECR: ".../togglmaster/evaluation-service:latest";
- Requests: CPU 50m, memória 128Mi; Limits: CPU 300m, memória 256Mi;

- Readiness: GET /health (delay 5s, timeout 1s, período 10s); Liveness: GET /health (delay 10s, timeout 1s, período 20s);
- Deployments confirmados com 2/2 réplicas prontas para auth-service, evaluation-service, flag-service e targeting-service; analytics-service em 0/0 (escalado a zero pelo KEDA);
- ServiceAccount dedicado por microsserviço (ex.: "evaluation-service-sa"), alinhado ao princípio do menor privilégio mencionado na Seção 8.5.

Estrutura de diretórios adotada:

k8s/

auth-service/ (deployment.yaml, service.yaml, configmap.yaml, secret.yaml)

flag-service/ (deployment.yaml, service.yaml, configmap.yaml, secret.yaml)

targeting-service/ (deployment.yaml, service.yaml, configmap.yaml, secret.yaml)

evaluation-service/ (deployment.yaml, service.yaml, configmap.yaml, secret.yaml)

analytics-service/ (deployment.yaml, service.yaml, configmap.yaml, secret.yaml, scaledobject.yaml)

Exemplo de Secret (sem valores reais): Secret do tipo "Opaque", nome "<serviço>-secret", chaves "DB_USER" e "DB_PASSWORD", referenciadas no Deployment via "envFrom.secretRef".

6. Ingress e Acesso Externo

CONCLUÍDO	Situação atual: Portas internas dos 5 Services, controller de Ingress, hostname do Load Balancer e Helm release confirmados via kubectl/console.
------------------	---

Rota planejada	Service de destino	Porta interna	Validação
<code>"/auth"</code>	<code>auth-service</code>	8001	Implementado — <code>auth-service</code> é somente interno, chamado via <code>GET /validate</code> por <code>flag-service</code> e <code>targeting-service</code> (não exposto pelo Ingress).
<code>"/flags"</code>	<code>flag-service</code>	8002	Confirmado — rota exposta pelo Ingress; teste HTTP 200 externo confirmado no console.
<code>"/targeting"</code>	<code>targeting-service</code>	8003	Implementado — a rota exposta pelo Ingress é <code>"/rules"</code> , roteada ao <code>flag-service</code> junto com <code>/flags</code> , <code>/validate</code> e <code>/admin</code> .
<code>"/evaluate"</code>	<code>evaluation-service</code>	8004	Confirmado — hot path (rota padrão) do Ingress; teste HTTP 200 externo confirmado no console.
<code>"/events"</code>	<code>analytics-service</code>	8005	Confirmado (não pendente) — o vídeo confirma que a ausência de rota HTTP externa é intencional: o <code>analytics-service</code> adota padrão de consumo de eventos, monitorando a fila SQS e processando apenas quando há mensagens, sem receber tráfego web direto.

Controller: `ingress-nginx` (Helm release `"togglemaster-ingress"`, namespace `"togglemaster"`); hostname do Load Balancer: `togglemaster-elb-1234567890.us-east-1.elb.amazonaws.com`; chamadas HTTP 200 externas confirmadas.

7. Escalabilidade Automática

CONCLUÍDO

Situação atual: Autoscaling confirmado via kubectl/CloudShell para os 5 microsserviços — HPA por CPU em auth-service, flag-service, targeting-service e evaluation-service; KEDA por profundidade de fila no analytics-service. Teste de carga do evaluation-service evidenciado (kubectl get hpa -w).

Todos os cinco microsserviços possuem HorizontalPodAutoscaler. Quatro (auth-service, flag-service, targeting-service e evaluation-service) escalam por CPU média; o analytics-service usa KEDA, escalando pela profundidade da fila SQS, incluindo a capacidade de escalar até zero réplicas quando não há mensagens pendentes.

Serviço	Métrica	Alvo	Réplicas min.	Réplicas max.	Resultado
<i>evaluation-service</i>	CPU média	60% de CPU	2	6	Confirmado por kubectl get hpa evaluation-service -w: sob carga, a CPU chegou a 712% do alvo e as réplicas escalaram de 2 para 6 (teto atingido), validando o funcionamento do HPA.
<i>analytics-service</i>	KEDA — profundidade da fila SQS (scale-to-zero)	Escalar ao detectar mensagens na fila (sem alvo percentual; orientado por número de mensagens)	0 (confirmado via kubectl get scaledobject: ScaledObject "analytics-service-sqs" com MIN 0/MAX 5; o HPA subjacente do KEDA reporta min 1 apenas como implementação interna)	5	Confirmado — ScaledObject "analytics-service-sqs" (trigger "aws-sqs-queue", MIN 0/MAX 5, READY true) e HPA "keda-hpa-analytics-service-sqs" com alvo de fila (<unknown>/5 em média); teste de carga do vídeo (15 mensagens) escalou o serviço de 0 para 1 pod, processou as mensagens e retornou a 0 réplicas.
<i>auth-service</i>	CPU média	70% de CPU	2	4	Confirmado — HPA ativo (kubectl get hpa), CPU média em ~1% do alvo, 2/2 réplicas em execução.
<i>flag-service</i>	CPU média	70% de CPU	2	4	Confirmado — HPA ativo (kubectl get hpa), CPU média em ~1% do alvo, 2/2 réplicas em execução.

Serviço	Métrica	Alvo	Réplicas min.	Réplicas max.	Resultado
<i>targeting-service</i>	CPU média	70% de CPU	2	4	Confirmado — HPA ativo (kubectl get hpa), CPU média em ~1% do alvo, 2/2 réplicas em execução.

Ferramenta de carga utilizada: comando “hey -z 5m -c 50 https://<hostname-do-elb>/evaluate” (5 minutos de teste, 50 conexões concorrentes), compatível com o comportamento observado no HPA do evaluation-service.

8. Desafios e Decisões Técnicas

8.1 Compatibilidade entre Unicorn e *setuptools*

Os três serviços Python entraram em reinício contínuo porque Unicorn 20.1.0 importava “*pkg_resources*”, indisponível na versão instalada de “*setuptools*”. A correção foi fixar “*setuptools==80.9.0*” no estágio de *build* e reconstruir as imagens sem cache.

8.2 Inicialização dos bancos PostgreSQL

Os bancos “*flags_db*” e “*targeting_db*” não existiam no PostgreSQL compartilhado porque scripts em “*/docker-entrypoint-initdb.d*” são executados apenas na primeira criação do volume. Foram criados os bancos e aplicados os schemas, e o script de inicialização foi documentado para tornar o ambiente reproduzível.

8.3 Simulação local dos serviços AWS

O LocalStack foi adotado para fornecer SQS e DynamoDB no mesmo contêiner. Os clientes AWS SDK receberam “*AWS_ENDPOINT_URL*” opcional: em desenvolvimento apontam para “*http://localstack:4566*”; em produção, sem essa variável, utilizam os endpoints reais da AWS.

8.4 Credenciais e configurações

O “*.env*” real não deve ser versionado. O repositório deve conter apenas “*.env.example*”, sem segredos. As credenciais “*test/test*” são exclusivamente locais e não devem ser reutilizadas na AWS.

8.5 Decisões futuras

As definições de limitações de IAM, escolha do controller de Ingress e estratégia de Secrets seguem as boas práticas recomendadas pela AWS e pelo Kubernetes (princípio do menor privilégio para roles/políticas do EKS, Ingress controller gerenciado e Secrets do Kubernetes/AWS Secrets Manager para valores sensíveis).

9. Considerações Finais e Links do Projeto

A etapa local demonstrou que os cinco microsserviços podem ser executados de forma integrada em contêineres e que o fluxo assíncrono de avaliação e analytics funciona sem dependência de recursos AWS reais. A próxima etapa consiste em reproduzir essa arquitetura com serviços gerenciados e implantar as aplicações no Amazon EKS.

Item	Link
Repositório (código-fonte + manifestos)	https://github.com/lucas-braga-dev/fiap_tc_fase_02
Vídeo de demonstração	https://youtu.be/d6hho-7MRGA

Item	Link
Badge Google Cloud Skills Boost (opcional)	Não aplicável — badge não gerado nesta entrega.

O provisionamento AWS foi concluído: os 5 microsserviços estão em execução no Amazon EKS (cluster “togglemaster”), com autoscaling validado via HPA/KEDA (incluindo teste de carga com escalonamento de 2 para 6 réplicas no evaluation-service e scale-to-zero no analytics-service) e os 5 repositórios ECR publicados. Identificadores de rede, endpoints de RDS/ElastiCache/DynamoDB/SQS e a configuração de Ingress/Load Balancer foram confirmados no console AWS.

Apêndice A - Controle de Progresso

Etapa	Status	Evidência / observação
Dockerfiles dos cinco serviços	Concluído	Build e execução local validados
Nove contêineres locais	Concluído	Todos os <i>health checks</i> retornaram status ok
Fluxo SQS -> DynamoDB local	Concluído	Evento persistido no LocalStack
“README” de reprodução local	Concluído	Passos, comandos e diagnóstico registrados
Repositório com os artefatos e documentação	Concluído	Documentações, artefatos, microsserviços e demais arquivos usados para implementação local. Disponível em: https://github.com/lucas-braga-dev/fiap_tc_fase_02
ECR e publicação de imagens	Concluído	5 repositórios confirmados no console AWS (ECR): auth, flag, targeting, evaluation e analytics-service.
Infraestrutura de dados AWS	Concluído	RDS (auth/flags/targeting), ElastiCache e endpoints/ARNs de DynamoDB/SQS confirmados no console — ver Seção 4.
Cluster EKS e Node Group	Concluído	Cluster “togglemaster” (Kubernetes 1.31), role, VPC/sub-redes e Node Group confirmados no console/CloudShell (ver Seção 4).
Manifestos Kubernetes	Concluído	Namespace, ConfigMap, requests/limits, probes e Secrets confirmados via kubectl para todos os microsserviços.
Ingress e Load Balancer	Concluído	Portas internas, controller (ingress-nginx), Helm release e hostname do Load Balancer confirmados via kubectl/console — ver Seção 6.
HPA / escalabilidade	Concluído	HPA confirmado nos 5 microsserviços (CPU nos 4 primeiros; KEDA por fila no analytics-service), com teste de carga evidenciado via kubectl get hpa -w.
Vídeo e links finais	Concluído	https://youtu.be/d6hho-7MRGA